

User as Code

Executable Memory for Personalized Agents

A novel paradigm where an agent's understanding of a user is maintained as a **modular, version-controlled software project**

Bojie Li

Chief Scientist, Pine AI

February 2026

Agenda

1. Introduction & Motivation
2. Background & Related Work
3. Three-Layer Evaluation Framework
4. Methodology: User as Code Architecture
5. Discussion
6. Conclusion

Part 1

Introduction & Motivation

The Memory Challenge for Personalized Agents

Building truly personalized, continuous-service AI agents requires a **User Memory** system.

Two Components of Memory

In Chinese, memory is 记忆 (Jìyì) — two characters that capture the two halves precisely:

记 (Jì) — Memorizing

Knowledge Extraction: extracting structured knowledge from raw interactions and persisting it

忆 (Yì) — Retrieval

Knowledge Retrieval: recalling the right knowledge at the right time to serve the user

User as Code contributes to **both**: structured code for memorizing, and executable retrieval via the generate-and-verify loop — the same code that stores knowledge serves as the substrate for computing answers no similarity search could produce.

Memory Layers

Working Memory — Single-session trajectory

Long-term Memory — Persistent cross-session knowledge ← **Our Focus**

Business State — Task-phase abstractions

Memory is not simply recording every word — it is gradually forming a **model of the user** through sustained interaction.

The "Bag of Facts" Problem

Current systems (VectorDBs, JSON stores, Knowledge Graphs) treat memory as a **loose collection of facts**.

Conflict Resolution Failure

Session 3: "I love cilantro" → **Session 12:** "I actually hate cilantro"

Vector retrieval might return both — or the wrong one.

Logic Gap

User has a flight on **Jan 15** and a passport expiring **Feb 18**.

LLMs struggle to detect: *180-day validity requirement means the passport is insufficient.*

The fundamental limitation: All existing formats — natural language, JSON, knowledge graphs — **separate representation from verification**. No amount of retrieval sophistication can reliably compute `(expiry - departure).days >= 180` .

The Representation Tension

Existing strategies trade **simplicity** for **expressiveness**:



What structured approaches *can* do

- Store `passport_expiry = "2025-02-18"`
- Tag with timestamps and entity IDs
- Perform CRUD operations

What they *cannot* do

- Express: *"if passport expires within 180 days of travel, raise alert"*
- Generate executable logic **on the fly** as novel situations arise
- Unify data and verification

The User as Code Proposal

Model a user's entire memory as a **self-evolving software project**.

Core Insight

The same Python that stores:

```
1 passport_expiry = date(2025, 2, 18)
```

can also express — and **execute**:

```
1 assert (passport_expiry - flight_date).days >= 180
```

No other format unifies storage and verification.

Architectural Fit

Modern AI agents (Claude, Cursor, OpenHands) are fundamentally **coding agents** built on virtual file systems.

- They read, write, and execute files
- The user project is a **directory in the agent's own workspace**
- No custom memory API needed

The Generate-and-Verify Loop: The LLM handles *semantic reasoning* ("should I check passport validity?"); the interpreter handles *computation* ("is $34 < 180$?").

Part 2

Background & Related Work

Current Approaches to User Memory

Approach	Mechanism	Strength	Weakness
Vector RAG	Semantic search over chunks	Basic recall	No conflict detection
Knowledge Graphs	Entity-relation triples	Structured relations	Rigid; no conditional logic
Mem0 / Memobase	CRUD + vector + LLM merge	Modular lifecycle	Still text-based verification
Contextual Retrieval	LLM context prepended to chunks	-67% retrieval failure	Retrieval only, no computation
Dual-Layer Memory	JSON Cards + Contextual RAG	Strongest baseline	Cannot express executable rules

State of the art (Dual-Layer): Advanced JSON Cards (structured core, always in context) + Contextual-Retrieval-enhanced RAG (on-demand details). But even this cannot compute: *"passport expires in 34 days — below 180-day requirement."*

Code as Knowledge Representation

Why Code?

- High-density, unambiguous representation
- Allows **Active Knowledge**: `if x: do y` is more robust than text
- Already the LLM's native output medium

The LLM + Interpreter Paradigm

ChatGPT Code Interpreter and sandboxed agents demonstrated LLMs can **generate code** for computation.

User as Code applies this paradigm to user memory.

Passive vs. Active Knowledge

```
1  # Passive (text/JSON) – LLM must reason
2  "Passport expires 2025-02-18,
3   flight departs 2025-01-15,
4   need 180 days validity"
5
6  # Active (code) – Interpreter verifies
7  days_left = (passport.expiry
8               - trip.departure).days
9  if days_left < 180:
10     alert(f"Only {days_left} days!")
11  # Output: "Only 34 days!" ✓
```

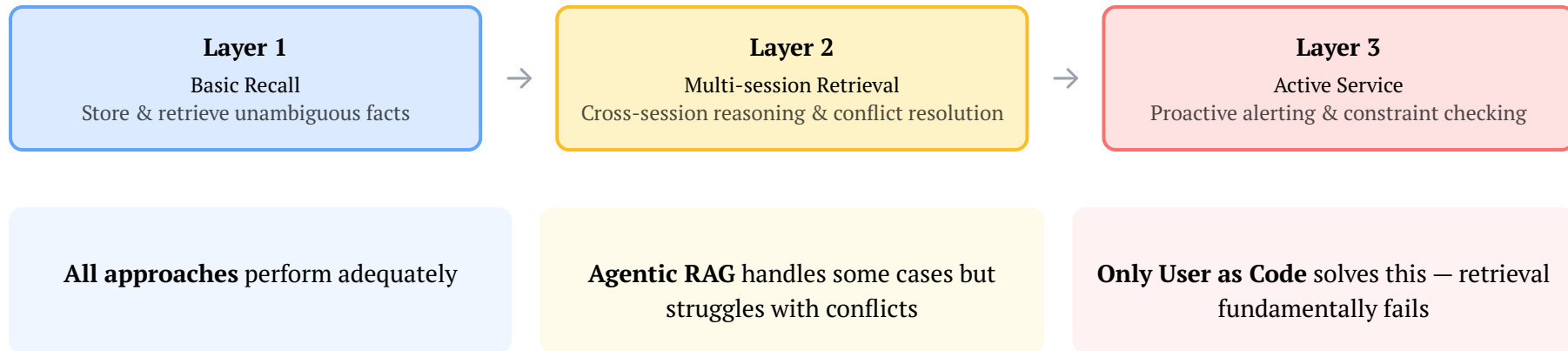
Turning unreliable natural-language arithmetic into **deterministic** verified computation.

Part 3

Three-Layer Evaluation Framework

Progressively Harder Memory Challenges

We propose a three-layer framework decomposing user memory capability into progressively harder levels.



Layer 1: Basic Recall

Task: Accurately store and retrieve directly provided, unambiguous information.

Example

"My membership number is 12345"

Later: "What's my membership number?" → **12345**

Result

All existing approaches perform adequately at this level. User as Code achieves **parity**.

Why It's Easy

- Single fact, single session
- No ambiguity or conflict
- Direct semantic match
- No computation required

This is the baseline — the "table stakes" of user memory. The real differentiation begins at Layer 2.

Layer 2: Multi-session Retrieval

Task: Retrieve and reason over information scattered across sessions, entities, and time periods.

Challenges

Disambiguation

Two cars mentioned across sessions: "*my car*" in session 3 vs. session 7 — which one?

Compound Events

Canceling "*the LA trip*" means flights + hotels + car rentals across multiple bookings.

Conflict Resolution

Multi-party conflicting instructions over time (Patricia says X, James says Y).

User as Code advantage: Single-source-of-truth state files with version-controlled patches provide deterministic conflict resolution. Each update carries a session reference for audit.

Layer 3: Active Service

Task: Proactively synthesize information across many sessions to provide **anticipatory, unsolicited help**.

The Challenge

Cross-reference a **new flight** (booked in Session 15) with **passport info** (stored in Session 2, months ago) to alert:

"Your passport expires Feb 18 — only 34 days of validity for your Jan 15 Tokyo trip. Most countries require 180 days."

Why Retrieval Fails

This is **arithmetic**, not a similarity match. No retrieval sophistication can compute:

$(2025-02-18) - (2025-01-15) = 34 \text{ days} < 180$

User as Code Solves This

```
1  # LLM generates this check:
2  for trip in upcoming_trips:
3      if trip.is_international:
4          days = (passport.expiry
5                  - trip.departure).days
6          if days < 180:
7              alert(
8                  f"Passport: {days} days "
9                  f"validity for {trip.dest}"
10                 f" — need 180!"
11             )
```

The interpreter **executes** it deterministically.

Result: "Passport: 34 days validity for Tokyo — need 180!"

Part 4

Methodology: The User as Code Architecture

A user is a self-evolving software project.

Design Principles

1. Separation of Structure and Data

Schema (class definitions) is separated from state (instance data) and archive (historical records).

2. Modularity by Life Domain

Memory is partitioned into independent domain packages — `travel/` , `finance/` , `health/` — each loadable and testable independently.

3. Progressive Disclosure

The agent navigates via a compact **manifest** (always loaded, ~200-300 tokens) and retrieves domain-specific modules on demand.

4. Unified Representation and Verification

Code serves as both the **storage format** the LLM reads and the **verification medium** the interpreter executes.

Application of the "**Infrastructure as Code**" philosophy to Agent Personalization

The User Project Structure

```
1  jessica_thompson/           # One user = one project
2  |─ manifest.py              # Compact index – ALWAYS in context
3  |─ domains/                 # Domain modules (one per life area)
4  |   |─ identity/
5  |   |   |─ schema.py        # PersonalInfo, ContactInfo class defs
6  |   |   |─ state.py         # Current name, DOB, contacts
7  |   |─ travel/
8  |   |   |─ schema.py        # Trip, PassportInfo class defs
9  |   |   |─ state.py         # Passport, trips, preferences
10 |   |─ finance/
11 |   |   |─ schema.py        # Account, Transaction class defs
12 |   |   |─ state.py         # Active accounts, pending transfers
13 |   |─ vehicles/
14 |   |   |─ schema.py        # Vehicle, MaintenanceSchedule class defs
15 |   |   |─ state.py         # Honda Accord, Tesla Model 3
16 |   |─ health/
17 |   |   |─ schema.py        # MedicalProfile, Allergy class defs
18 |   |   |─ state.py         # Allergies, current medications
19 |   |─ family/
20 |   |   |─ schema.py        # FamilyMember, Relationship class defs
21 |   |   |─ state.py         # Husband James, Daughter Sarah (8)
22 |─ constraints/             # Persistent cross-domain constraints
23 |   |─ travel_readiness.py   # LLM-generated, promoted from ad-hoc
24 |   |─ financial_authorization.py # LLM-generated, promoted from ad-hoc
25 |   |─ health_safety.py      # LLM-generated, promoted from ad-hoc
```

Key Structural Properties

Bounded Schema, Unbounded Data

- Domain modules are bounded (~10-20)
- `state.py` holds only **current active state**
- History lives in the archive (Tier 3)

Cold Start

For a new user, the agent **bootstraps** from the first conversation:

1. Create initial domains
2. Populate schema and state
3. Generate the first manifest

The structure **emerges organically** from the user's actual life.

Living Architecture

Domains are **added, refactored, and retired** — not hardcoded.

```
1  # The agent might:
2  # 1. Create a new domain
3  mkdir("domains/real_estate/")
4  # 2. Split a growing domain
5  # vehicles/ → vehicles/ + boats/
6  # 3. Merge sparse domains
7  # combine hobbies/ + sports/
8  # 4. Archive stale state
9  # move old trips to archive
```

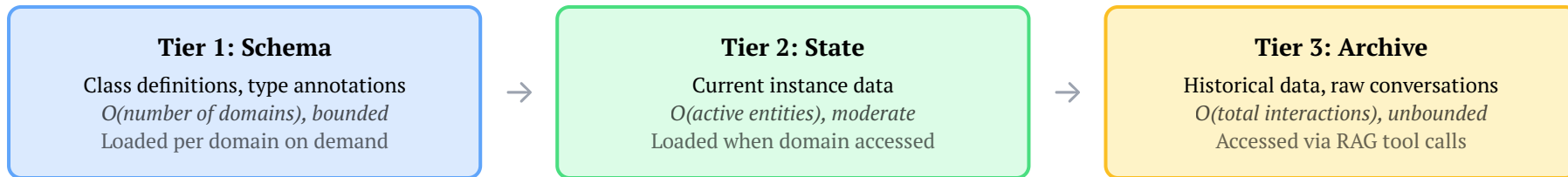
The agent performs **periodic revision**: schema evolution, domain splitting/merging, stale state archival.

Memory Organization: Five Types

Memory Type	Stored As	LLM Interaction	Example
Factual State	Typed attributes in <code>state.py</code>	Read	<code>passport_expiry = date(2025, 2, 18)</code>
Subjective Preferences	String fields in <code>state.py</code>	Read (not computable)	<code>seat_notes = "Aisle on long flights..."</code>
Persistent Constraints	Functions in <code>constraints/</code>	Auto-Execute after updates	<code>travel_readiness.check(project)</code>
Ad-hoc Verification	LLM-generated at query time	Generate-and-Execute	<code>print((passport_expiry - trip.date).days)</code>
Narrative Context	Conversation chunks in archive	Search via RAG	<code>archive.search("why Jessica chose JAL")</code>

The Key Distinction: *Read path* vs. *Verify path* — **Subjective preferences** → read-only, LLM reasons contextually | **Factual relationships** → verified by interpreter, deterministic arithmetic

Memory Organization: Three Tiers



Tier 2 Example: `domains/travel/state.py`

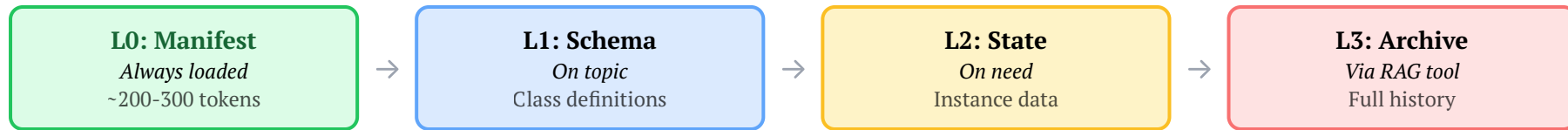
```
1 passport = PassportInfo(number="AB*****67", expiry=date(2025, 2, 18), country="US")
2 upcoming_trips = [
3     Trip("Tokyo", date(2025, 1, 15), date(2025, 1, 25), refs=["JAL-9823"], intl=True),
4     Trip("Mexico City", date(2025, 3, 10), date(2025, 3, 17), refs=["AA-4561"], intl=True),
5 ]
6 seat_notes = "Prefers aisle on flights > 6hrs, window otherwise. Always window on Japan routes."
```

Progressive Disclosure via the Manifest

The `manifest.py` is **always in the agent's context** (~200-300 tokens):

```
1  # manifest.py - ALWAYS IN AGENT CONTEXT
2  """User: Jessica Thompson | user_12345 | Updated: 2025-02-14T10:30:00Z"""
3
4  DOMAINS = {
5      "identity": "Core profile, contacts | updated 2025-01-20",
6      "travel":   "3 upcoming trips | passport expires 2025-02-18",
7      "finance":  "2 accounts, 1 pending wire transfer",
8      "vehicles": "Honda Accord (service Fri), Tesla Model 3",
9      "health":   "Allergies: peanuts, penicillin | Rx: cetirizine",
10     "family":   "Husband: James | Daughter: Sarah (8, eczema)",
11 }
12
13 ACTIVE_ALERTS = [
14     "[CRITICAL] travel_readiness: Passport expires 2025-02-18, "
15     "Mexico City trip departs 2025-03-10 (passport EXPIRED at departure)",
16     "[CRITICAL] travel_readiness: Passport expires 2025-02-18, "
17     "Tokyo trip departs 2025-01-15 (only 34 days validity, need 180)",
18     "[WARNING] financial_auth: Conflicting wire transfer instructions "
19     "from Patricia (session 2025-02-10) and James (session 2025-02-12)",
20 ]
```

Four Levels of Disclosure



Self-Documenting System

The manifest doubles as the agent's **bootstrap instruction**.

By reading it, the agent understands:

- Project layout
- Available domains
- Active concerns and alerts

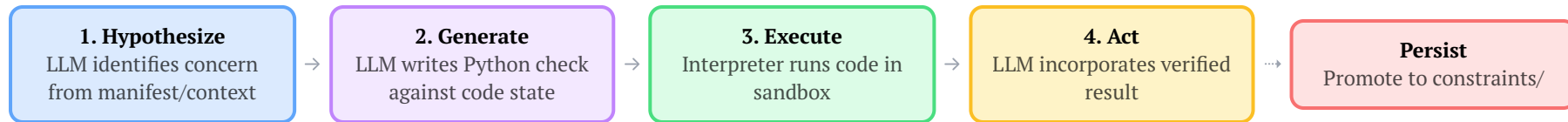
No Separate Instructions Needed

No "how to use this memory system" prompt is needed.

The project **is its own documentation** — just as a well-structured codebase is navigable from its directory layout.

The Generate-and-Verify Loop

The core mechanism distinguishing User as Code from retrieval-based approaches.



Division of Labor

	LLM	Interpreter
Role	Semantic reasoning	Computation
Decides	<i>What</i> to check	<i>Correctness</i> of check
Example	"Check passport?"	"Is 34 < 180?" → <code>True</code>

From Ad-hoc to Persistent

When an ad-hoc check proves useful:

1. Agent writes it to `constraints/`
2. Stored as `def check(project) -> List[Alert]`
3. **Auto-evaluates** after relevant state updates
4. Alerts surface in `ACTIVE_ALERTS`

Constraints are **not pre-authored** — they emerge from the LLM's own reasoning.

The Write Path: Update Pipeline



Two-Layer Persistence

- **Working Directory** (session-scoped): Drafts, ad-hoc checks, schema experiments — ephemeral
- **Persistent Storage** (cross-session): Committed project — source of truth

Autonomous save: The agent decides when to commit, like a developer choosing when to `git commit`.

No Bespoke Toolset

The agent reads, writes, and executes files — **the same primitives any coding agent already has.**

- Validation pipeline lives inside the repo
- Inspectable and version-controlled
- Only specialized infra: RAG for Tier 3

Every patch committed with a **source-session reference** for audit trail.

Periodic Revision & External Knowledge

Periodic Revision

Beyond incremental patches, the agent performs **holistic revisions**:

- Schema evolution and restructuring
- Domain splitting / merging
- Stale state archival
- Constraint pruning
- Cross-domain reference audits
- Preference synthesis

Triggered by: agent judgment, scheduled cycles, or thresholds.

External Knowledge Integration

Constraints can import **versioned factual reference packages**:

```
1  # constraints/travel_readiness.py
2  from refs.visa_policy import VisaPolicy
3  from refs.passport_rules import PassportRules
4
5  def check(project) -> List[Alert]:
6      policy = VisaPolicy.for_country(
7          trip.destination)
8      min_days = policy.passport_validity_days
9      # Grounded in verified, current
10     # regulations – not stale training data
```

Analogous to RAG grounding — not hand-crafted features.

Part 5

Discussion

The Bitter Lesson and the Role of Code

Objection: *Typed schemas contradict Sutton's "Bitter Lesson" — hand-crafted structure is eventually superseded by general methods leveraging computation.*

What is LLM-Generated (Domain Knowledge)

- Schemas, constraints, tests, state data
- Domain partitioning decisions
- `class Trip` design and field selection
- Constraint creation and promotion
- Periodic restructuring

This is *learned* structure — the model's own decisions, not imposed features.

What is Human-Designed (Infrastructure)

- Project directory convention
- Update pipeline
- Progressive disclosure levels
- Generate-and-verify loop

This is *engineering infrastructure* — analogous to choosing Git or CI/CD. Tools, not domain features.

The interpreter is a **tool**, not a **rule**. The LLM retains full freedom in *what to check*; the interpreter guarantees correct computation.

The Formalization Boundary

Factual / Computational

Typed attributes · Date arithmetic · Cross-domain constraints

← User as Code excels



Soft memory as strings →

Hard-to-Formalize

Context-dependent preferences · Behavioral patterns ·
Emotional context

Hard-to-Formalize Memory

Lives as **string annotations** — benefiting from structure and version control, but not executable verification.

```
1 # Soft memory example
2 seat_notes = """Prefers aisle on flights
3 > 6hrs, window otherwise. Exception:
4 always window on Japan routes
5 for Mt. Fuji view."""
```

A Unified Home

User as Code provides a **unified home** where both types coexist:

- Factual state → typed, verified
- Soft preferences → annotated strings
- Both benefit from version control, domain organization, and progressive disclosure

Future work: LLM-synthesized personality profiles periodically regenerated from cross-domain review.

Agent-Native Architecture & Limitations

Agent-Native Architecture

The user project is a **directory in the agent's virtual file system**.

- Reads, writes, and executes files
- Same primitives the agent already has
- **No custom memory API**
- **No special toolset**
- **No new infrastructure** (except RAG for archive)

Saving memory is an **autonomous agent decision**, not a system hook. **Trivially adoptable** by any coding agent framework.

Limitations

1. Verification Overhead Generating structured code and verifying it (tests, type checks) is more expensive than appending text. *Trade-off: pay at write time for correctness at read time — a favorable exchange.*

2. Constraint Logic Errors Interpreter guarantees correct *computation* but not correct *logic* (e.g., wrong threshold). Same failure mode as any LLM judgment.

3. LLM Code Generation Quality Mitigated by strict type checking, test validation, and reject-and-retry loops.

Part 6

Conclusion

Why Code — and Nothing Else

The core claim is not that code is a *nice* format for user memory. It is that code is the **only** format where the agent can seamlessly transition from reading a user's state to writing and executing checks against it — with no translation step.

Natural Language / Markdown

Read ✓ Verify ✗

The LLM must *reason* about "34 < 180" in prose — and gets it wrong.

JSON / Knowledge Graph

Read ✓ Verify requires translation

Data must be parsed, loaded into code, *then* checked — friction + error surface.

Python (User as Code)

Read ✓ Execute ✓ **Same file**

The agent reads `passport.expiry` and writes `assert (expiry - date).days >= 180` — zero translation.

What This Enables

- **Generate-and-verify loop:** LLM reasons *what* to check; interpreter guarantees the *computation* — no hallucinated arithmetic
- **Emergent constraints:** ad-hoc checks promoted to persistent monitors, autonomously — no human authoring
- **Active Service (Layer 3):** the only approach that solves proactive alerting, because it's arithmetic, not retrieval

Architecture in One Sentence

A user is a **self-evolving software project** in the agent's own workspace — manipulated with the same file and interpreter primitives the agent already has, structured by three-tier progressive disclosure, and kept consistent by version-controlled patches and executable tests.

Thank You

User as Code: Executable Memory for Personalized Agents

*Code closes the gap between knowing and computing —
the same Python the agent reads is the Python the interpreter executes.*

Bojie Li

Chief Scientist, Pine AI

Powered by  Slidev